// Codechum base



Introduction to Java

Java

> a versatile and widely used programming language
> first released by Sun Microsystems in 1995.
> developed by James Gosling and his team

- **Reasons:** to address the complexities of C and C++
- providing a platform-independent and object-oriented approach to software development

> became one of the most popular languages for building various types of applications

- Web
- Mobile
- Desktop
- Enterprise solutions

> Java is a general-purpose programming language that follows the "Write Once, Run Anywhere" (WORA) principle

- means that Java programs can be compiled into platform-independent bytecode, which can then be executed on any device with a Java Virtual Machine (JVM) installed
    - The JVM acts as an interpreter that translates the bytecode into machine code specific to the underlying operating system.

> Java is also known for its object-oriented programming (OOP) paradigm

- encourages the use of classes and objects to model real-world entities and their interactions
    - this approach promotes code reusability, modularity, and maintainability

Simple Java code



Note: public class Main defines a class named "Main". The main() method is the entry point of a Java program, and System.out.println() is used to display output.

Features of Java

Platform Independence

> one of its most significant advantages
> By compiling Java source code into bytecode, it can run on any device with a compatible JVM

- allows developers to write applications that can be deployed on various platforms without modification

Object-Oriented Programming (OOP)

> Java is fully object-oriented, meaning everything in Java is an object
> It supports principles such as encapsulation, inheritance, and

polymorphism, allowing developers to create well-structured and reusable code

```java
class Rectangle {
    private int width;
    private int height;

    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }

    public int getArea() {
        return width * height;
    }
}

```

Robust and Secure

> Java was designed with a focus on robustness and security.
> It includes features like **automatic memory management (garbage collection)** to prevent memory leaks and runtime exceptions, providing a more stable environment for applications.

Rich Standard Library

> Java comes with a comprehensive standard library that provides a wide range of classes and methods for performing common tasks.
> The Standard Library simplifies development by offering ready-to-use functionality for tasks such as input/output, networking, data manipulation, and more.

```java
import java.util.Scanner;

public class InputExample {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter your name: ");
        String name = scanner.nextLine();
        System.out.println("Hello, " + name + "!");
    }
}

```

Multithreading and Concurrency

> Java supports multithreading and concurrency, allowing developers to create applications that can perform multiple tasks simultaneously.

- This is particularly useful for applications that require handling multiple operations concurrently, such as web servers and GUI applications.

Performance and Portability

> Java is an interpreted language, but advancements in JVM technology have made it highly performant.

- Java's portability allows applications to be easily migrated and run on different platforms with minimal changes.

Applications of Java

> **Web Development:** widely used for building web applications and dynamic websites using technologies such as JavaServer Pages (JSP), Servlets, and frameworks like Spring and JavaServer Faces (JSF).
> **Mobile Applications:** has been the traditional programming language for Android app development, though

Kotlin has become increasingly popular in recent years.
> **Enterprise Software:** is a common choice for developing large-scale, enterprise-level applications due to its scalability, reliability, and extensive libraries.
> **Desktop Applications:** Java Swing and JavaFX are used for creating desktop applications with graphical user interfaces (GUIs).
> **Cloud-Based Applications:** frequently used in cloud-based applications and services due to its ability to scale and its cloud-friendly ecosystem.
> **Big Data and Analytics:** used in big data processing frameworks like Apache Hadoop and Apache Spark, as well as in data science and analytics applications.
> **Internet of Things (IoT):** Java's ability to run on various hardware platforms and its support for embedded systems make it suitable for IoT development.



Basic Structure

Why?

> The structure of a Java program provides an organized framework for writing code and defines how the program functions.

- It ensures that the code is well-structured, readable, and maintainable.

> The main method, in particular, serves as the entry point of a Java program and controls the program's execution flow.
> Understanding the basic structure and the role of the main method is fundamental to writing Java programs.

Applications of Java

```
1  // Package declaration (optional)
2  package com.example.myapp;
3
4  // Import statements (optional)
5
6  // Class definition
7  public class MyClass {
8      // Main method
9      public static void main(String[] args) {
10         // Main method body
11
12     }
13 }
14
```

Package Declaration (optional) (line 2)

> A Java program can be organized into packages, which are used to group related classes together.
> The package declaration, if present, should be the first line of code in a Java file.

- Specifies the package to which the class belongs.

> Packages are **optional**, but they are essential for organizing large projects and avoiding naming conflicts.

Import Statements (optional) (line 4)

> They allow you to access classes and other elements from external packages without specifying their fully qualified names.

> Import statements, if used, follow the package declaration and precede the class definition.

- If no import statement is present, Java uses classes from the default **java.lang** package automatically.

> *Examples:*

- import java.util.ArrayList;
- import java.util.*;
- import static java.lang.Math.PI;
- import static java.lang.Math.sqrt;
- import module java.base;

Class Definition (line 7)

> A Java program typically consists of one or more classes.
> The class definition contains the code that defines the behavior and properties of objects belonging to that class.
> A Java source file can contain only **one** public class, and the filename should **match** the name of the public class.

- If there is no public class, the filename can be anything.

Main Method and Program Execution Flow (line 9)

> serves as the entry point of a Java program.
> It has a specific signature, and the Java Virtual Machine (JVM) looks for this method to start the program's execution.

```
public static void main(String[] args) {

    }
```

> The main method must have the following signature:

1. The access modifier must be public (as the JVM needs to access it from outside the class).
2. The return type is void, indicating that the method does not return any value.
    - Static is a modifier but it partners with void.
3. The method name must be main.
4. The method(data type) must take an array of strings as its parameter.
    - This array, commonly named args, holds any command-line arguments passed to the program. Command-line arguments can be used to pass data to the program when it starts.

> The program execution flow follows the statements inside the main method from top to bottom.

- Additional methods can be called from the main method or from other methods defined in the program.
- By organizing code into methods, the program's logic can be modularized, making it easier to understand, maintain, and debug.



Libraries

Imports

> They provide the necessary information to the compiler about the classes and libraries used in a program.
> They serve as a way to access predefined classes and packages, as well as to include custom classes and packages created by developers.
> Importing the appropriate classes and packages is essential for utilizing the functionalities provided by Java's standard library and other external libraries.

Importing Classes and Packages

> To import a class or package in a Java program, the import statement is used.

- The import statement tells the compiler where to find the class or package used in the program.

> *Example:*

```java
import java.util.Scanner;

public class MyClass {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter your name: ");
        String name = scanner.nextLine();
        System.out.println("Hello, " + name + "!");
        scanner.close();
    }
}

```

- Here's an example of importing the java.util package, which provides utility classes like Scanner for input operations. In the above code, the import java.util.Scanner; statement is placed before the class definition. It tells the compiler to import the Scanner class from the java.util package, enabling the use of Scanner for input operations.

Commonly Used Import Statements in Java Programming

java.util Package (Utility Classes)

> Provides utility classes that are commonly used for input, output, data manipulation, and other general-purpose functionalities.
> Examples include:

- Scanner for input operations
- ArrayList for dynamic arrays
- Random for generating random numbers.

```java
import java.util.Scanner;
import java.util.ArrayList;
import java.util.Random;

```

java.io Package (Input/Output)

> Contains classes for performing input and output operations.
> It includes classes like:

- File
- FileInputStream
- FileOutputStream
    - which are used for reading and writing files.

```java
import java.io.File;
import java.io.FileInputStream;
import java.io.FileOutputStream;

```

java.awt Package (Abstract Window Toolkit)

> Provides classes for creating graphical user interfaces (GUI) in Java applications.
> It includes classes for:

- Windows
- Buttons
- Graphics
- Event handling

```java
import java.awt.Frame;
import java.awt.Button;
import java.awt.Graphics;

```

java.text Package (Text Formatting)

> Offers classes for formatting and parsing text.
> It includes classes like:

- DateFormat
- NumberFormat
    - for date, time, and number formatting.

```java
import java.text.DateFormat;
import java.text.NumberFormat;

```

java.time Package (Date and Time)

> Provides classes for working with date and time in Java.
> It includes classes like:

- LocalDate
- LocalTime
- DateTimeFormatter

```java
import java.time.LocalDate;
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;

```



Symbols

Variables

> They are named storage locations that hold values during program execution.
> They are used to store and manipulate data.
> In Java, variables must be declared with a specific data type before they can be used.
> Common data types in Java include:

- int
- float
- char
- double
- String.

//String lang ang naka capital sa una

Declaring and Using Variables

> To declare a variable in Java, you specify its data type followed by the variable name.

> In the above code, an integer variable named age is declared. Once declared, you can assign a value to the variable using the assignment operator (=).

```
int age;
Age = 20;
```

> You can also declare and assign a value to a variable in a single statement

```
int age = 20;
```

> Variables can be used in expressions, assignments, and other operations throughout the program.

Constants

> They are fixed values that do not change during program execution.
> They are useful for storing values that remain constant throughout the program's execution.
> In Java, constants are declared using the final keyword.

Declaring and Using Constants

> In Java, constants are typically declared using the final keyword followed by the data type.

```
final int MAX_VALUE = 100;
```

- In the above code, a constant integer named MAX_VALUE is declared with a value of 100.
- Once declared, the constant cannot be modified during program execution.

> Constants are often used to define meaningful names for fixed values and to improve code readability.

```
final float PI = 3.14159;
```

Methods (Functions)

> Methods in Java are blocks of code that perform specific tasks.
> They are used to organize code into logical units, improve code reusability, and enhance program modularity.
> Methods in Java consists:

- method signature (return type and name)
- parameters (optional)
- method body (code block).

Declaring and Using Methods

> To declare a method in Java, you specify the return type, method name, and any parameters it accepts.

```
public int addNumbers(int a, int b) {
        int sum = a + b;
        return sum;
    }
```

- In the above code, a method named addNumbers is declared, which takes two integer parameters a and b. The method calculates their sum and returns the result.

> Methods can be called (invoked) from other parts of the program.

- To call a method, you use its name followed by parentheses and any required arguments.

int result = addNumbers(5, 3);

- In the above code, the addNumbers method is called with arguments 5 and 3. The returned value is assigned to the variable result.

> Methods play a crucial role in Java programming as they allow for code organization, reusability, and the separation of complex tasks into smaller, manageable units.



Comments

Importance of Comments in Java Code Documentation

> Comments play a crucial role in code documentation in the Java programming language.
> They provide additional context, explanations, and instructions within the code, aiding both developers and future maintainers.

Enhancing Code Readability and Understanding

> They act as a **form of communication** within the code, helping developers understand the purpose, functionality, and logic of different sections of code.
> They provide **clarity** by explaining complex algorithms, outlining key steps, and documenting important decisions made during the development process.
> Well-placed comments make code **more readable and easier to understand**, reducing the time and effort required for comprehension.

Facilitating Collaboration and Maintenance

> Comments also foster **collaboration** among team members by facilitating code sharing and maintenance.
> They enable multiple developers to work together on a project, understand each other's code, and make changes or improvements.
> They serve as a **documentation resource**, allowing developers to follow code logic, identify potential issues, and make informed modifications or updates.

Assisting Debugging and Troubleshooting

> During the debugging process, comments can be **invaluable** in identifying and isolating problematic code sections.
> By temporarily commenting out specific code blocks or providing explanations about the intended functionality, developers can narrow down the root cause of an issue and accelerate the troubleshooting process.

Different Types of Comments in Java

Single-Line Comments

> They are used to add explanations or comments on a single line. They are typically used for short descriptions or annotations.

> In Java, single-line comments start with // and continue until the end of the line.

```
int age; // Variable to store the user's age
```

Multi-Line Comments

> Multi-line comments, also known as **block comments**, allow for longer explanations or comments spanning multiple lines.
> They are useful for documenting larger sections of code, outlining methods, or providing more extensive explanations.
> In Java, multi-line comments start with /* and end with */.

```
/*
        This method calculates the sum
        of two numbers.
        Parameters:
        - a: First number
        - b: Second number
        Returns:
        The sum of a and b.
*/
public int addNumbers(int a, int b) {
            return a + b;
        }
```

Commenting Out Code

> Comments can also be used to temporarily disable or "comment out" sections of code.
> This technique is helpful during testing, experimentation, or when wanting to exclude specific code without deleting it.

> By commenting out code, you ensure that it is not executed while still keeping it in the source file for future reference or reintegration.

```
/*
        int x = 5;
        int y = 10;
        int sum = x + y;
*/
```



Variables

Declaration and Initialization of Variables

> Variables are essential components in programming that allow us to store and manipulate data.
> They play a crucial role in defining and managing information within a Java program.

Declaration of Variables

> Variables must be declared before they can be used. A variable declaration consists of specifying the data type and the variable name.

```
int age;
```

- In the above code, we declare an integer variable called age. This declaration informs the compiler about the type of data that age will hold.

Initialization of Variables

> After declaring a variable, we can assign an initial value to it through initialization.
> **Initialization** is the process of providing an initial value to a variable at the time of declaration.

```
int age = 25;
```

- In the above code, we declare and initialize the variable age with the value 25. Initialization allows us to set an initial value, making the variable ready for use immediately.

Usage of Variables

Assigning Values to Variables

> Once declared and initialized, variables can be assigned new values using the assignment operator (=).

```
age = 30;
```

Using Variables in Expressions

> Variables can be used in expressions to perform calculations or manipulate data.

```
int result = age + 5;
```

Printing Variables

> To display the value of a variable, we can use the System.out.println() method in Java.

```
System.out.println("The current age is " + age);
```

Variable Naming Conventions

> To write clean and readable code, it is essential to follow certain naming conventions for variables.
> Here are some common practices:

- Use descriptive names that reflect the purpose or meaning of the variable.
- Start variable names with a lowercase letter and use camel case for multiple words (e.g., firstName, studentAge).
- Avoid using reserved keywords as variable names.
- Use meaningful abbreviations when necessary to maintain readability.

> By following consistent naming conventions, code becomes more understandable, maintainable, and easier to collaborate on.



Data Types

Primitive Data Types in Java

> Java has two categories of data types: **primitive data types and reference data types**.
> Primitive data types are basic data types provided by the language, and they are not objects.

- They are used to represent simple values.

Type	Description	Size (in bytes)	Range
byte	Used to represent small integers	1	-128 to 127
short	Used to represent small integers	2	-32,768 to 32,767
int	Used to represent whole numbers	4	-2,147,483,648 to 2,147,483,647
long	Used to represent large integers	8	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	Used to represent decimal numbers	4	Approximately ±3.40282347E+38F (6-7 significant decimal digits)
double	Used to represent decimal numbers	8	Approximately ±1.79769313486231570E+308 (15 significant decimal digits)
char	Used to represent individual characters	2	'\u0000' (or 0) to '\uffff' (or 65,535 inclusive)
boolean	Used to represent true or false	1	true or false

- These data types are used to declare variables, specifying the type of data the variable can hold.

Non-Primitive Data Types (Reference Types)

> Java also has non-primitive data types, which are also called reference types. These include arrays, classes, interfaces, etc.
> They are used to create objects and provide more complex data structures.

- Unlike primitive data types, reference types are created using the new keyword or by assigning them to existing objects.

Declaring and Initializing Variables

> Java requires that you explicitly specify the data type when declaring variables.

```
1  // Declaration of variables
2  int age;
3  float weight;
4
5  // Initialization of variables
6  age = 25;
7  weight = 65.5f;
8
```

```
1  int age = 25;
2  float weight = 65.5f;
3
```

Character Data Type

> The char data type represents a single 16-bit Unicode character.
> It is used to store individual characters such as letters, digits, or symbols.
> Unicode characters allow Java to support a wide range of international characters and symbols.

char grade = 'A';

Numeric Data Types

Integer Data Types

> Java provides four integer data types to represent whole numbers:

- byte
- short
- int
- long
    - The int data type is the most commonly used for whole numbers.

```
int population = 1000000;
long veryLargeNumber = 1234567890123456789L; // Note the 'L' suffix to denote a long literal
```

Floating-Point Data Types

> Java provides two floating-point data types to represent decimal numbers with different levels of precision: float and double.
> The double data type is the most commonly used for decimal numbers.

```
float pi = 3.14f; // Note the 'f' suffix to denote a float literal
double largeNumber = 1234567890.123456789;
```

Boolean Data Type

> Represents a boolean value, which can be either true or false.
> It is often used for conditional statements and decision-making.

```
boolean isRaining = true;
```



Constants

> Constants in Java are fixed values that do not change during the execution of a program.
> They are used to store data that remains constant throughout the program's lifecycle.

Numeric Constants

> Numeric constants in Java represent fixed numeric values, either integers or floating-point numbers.

Integer Constants

> Integer constants are whole numbers without fractional parts.
> They can be represented in decimal, octal, or hexadecimal notation.

```
int decimal = 10;       // Decimal representation (base 10)
int octal = 012;        // Octal representation (base 8)
int hexadecimal = 0xA;  // Hexadecimal representation (base 16)

```

Floating-Point Constants

> Floating-point constants represent real numbers with fractional parts.
> They can be expressed in decimal or exponential notation.

```
float decimalFloat = 3.14f;      // Decimal representation (suffix 'f' indicates float)
double decimalDouble = 3.14;     // Decimal representation (double precision)
float exponentialFloat = 2e-3f;  // Exponential representation (2 * 10^-3)

```

- In the above code, we declare floating-point constants using decimal and exponential notation in Java.
- Note that for float, we use the suffix 'f' to indicate it's a float type.

Character Constants

> Character constants represent individual characters enclosed in single quotes.

```
char letter = 'A';   // Character constant representing the letter 'A'
char symbol = '$';   // Character constant representing the symbol '$'

```

String Constants

> String constants are sequences of characters enclosed in double quotes.

```
String greeting = "Hello, world!";  // String constant representing a greeting

```

Usage of Constants

> Constants can be used in various parts of a Java program, such as assignments, expressions, and function arguments.

```
final float PI = 3.14159f;    // Declaration of a constant variable using 'final' keyword
int radius = 5;
float circumference = 2 * PI * radius;   // Usage of the constant in an expression

```

- In the above code, we declare a constant variable PI using the final keyword, and we use it in an expression to calculate the circumference of a circle in Java.
- The final keyword ensures that the value of the constant cannot be changed once assigned.



Output Operations

> Output operations are an essential part of programming as they allow programs to display information to the user.
> In Java programming, displaying output to the screen or console can be achieved using the System.out.println() method.

Displaying Output to the Screen/Console

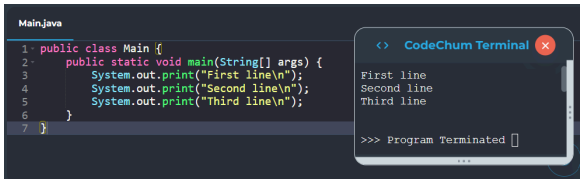
> To display output to the screen or console in Java, you can use the System.out.println() method.
> This method is part of the System class and allows you to output text, variables, and other data to the screen.
> It automatically adds a newline character at the end, so each call to println() starts a new line.



Using \n for Newlines

> In Java, the \n character is used to insert a newline in the output.
> This special character moves the cursor to the beginning of the next line, allowing you to format your output text neatly.

- While System.out.println() automatically appends a newline character, you can use \n within strings for additional newlines.



Using System.out.println() for Formatted Output

> While System.out.println() automatically adds a newline character, if you need to display output without a newline, you can use System.out.print().

- However, Java does not have format specifiers like C's printf, but you can achieve similar formatted output using the **concatenation (+) and conversion methods**.



- In the above code, the + operator is used to concatenate the strings and variables.
- For formatting the floating-point number num2, we use String.format("%.2f", num2), where %.2f indicates that we want two decimal places for the double value num2.

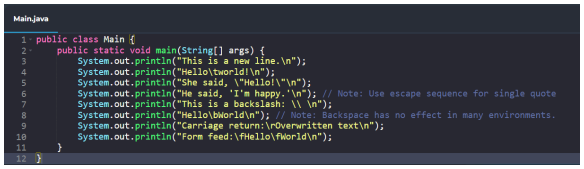


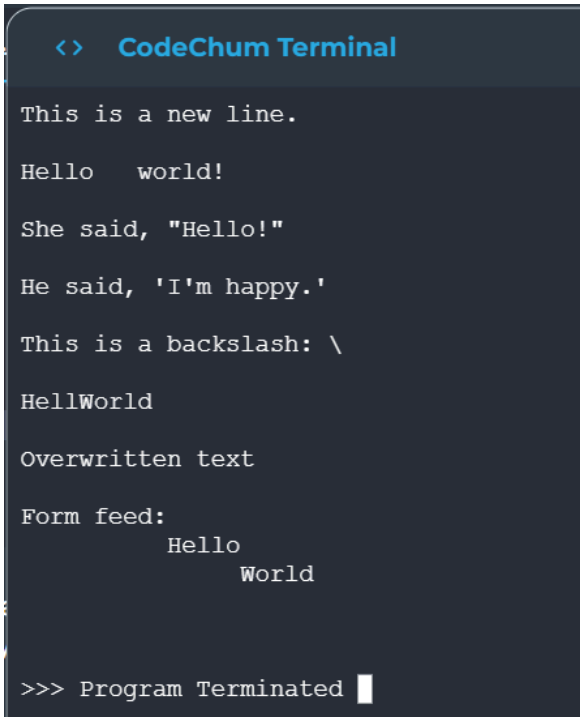
Basic Escape Sequences

> Escape sequences are special character combinations that allow you to represent characters that are difficult to type or have a special meaning within a string.
> In Java programming, escape sequences are used to insert characters that cannot be typed directly into a string.

Common Escape Sequences

> **\n: Newline** - Moves the cursor to the beginning of the next line.
> **\t: Tab** - Inserts a horizontal tab.
> **\": Double quote** - Inserts a double quote character.
> **\': Single quote** - Inserts a single quote character.
> **\\: Backslash** - Inserts a backslash character.
> **\b: Backspace** - Moves the cursor back one position.
> **\r: Carriage return** - Moves the cursor to the beginning of the current line.
> **\f: Form feed** - Moves the cursor to the next logical page.







Arithmetic Operators

> They are fundamental elements of any programming language, including Java. They allow you to perform mathematical operations on numeric values.
> In Java, the basic arithmetic operators include addition (+), subtraction (-), multiplication (*), division (/), and modulus (%).
> Understanding how to use these operators correctly and being aware of operator precedence and associativity is crucial for writing effective and accurate mathematical expressions in your Java programs.

Basic Arithmetic Operators in Java

> **Addition (+):** Adds two values together.
> **Subtraction (-):** Subtracts one value from another.
> **Multiplication (*):** Multiplies two values.
> **Division (/):** Divides one value by another.
> **Modulus (%):** Computes the remainder after division.
> **Increment (++):** Increases the value of a variable by 1.
> **Decrement (--):** Decreases the value of a variable by 1.



Operator Precedence and Associativity in Java

> **Operator precedence** determines the order in which operators are evaluated in an expression.

- Operators with higher precedence are evaluated first. When multiple operators have the same precedence, the associativity of the operators comes into play.

> **Associativity** determines the order in which operators of the same precedence are evaluated, either from left to right or right to left.

1. Unary operators (e.g., ++, --)
2. Multiplication (*), Division (/), Modulus (%)
3. Addition (+), Subtraction (-)

> To avoid confusion and ensure the correct evaluation of expressions, **you can use parentheses to explicitly specify the desired order of operations**.



Assignment Operators

> Assignment operators are used to assign values to variables in programming languages, including Java.
> They provide a concise and efficient way to update the value of a variable based on a computation or another variable's value.

Different Assignment Operators in Java

> **Basic Assignment Operator (=):** The basic assignment operator is used to assign a value to a variable.
> It assigns the value on the right-hand side to the variable on the left-hand side.

Main.java
```java
public class Main {
    public static void main(String[] args) {
        int a = 1;
        System.out.println("a: " + a); // Output will be "a: 1"
    }
}
```

Compound Assignment Operators

> **Addition Assignment Operator (+=):** It adds the value on the right-hand side to the variable on the left-hand side and assigns the result to the left-hand side variable.

Main.java
```java
public class Main {
    public static void main(String[] args) {
        int num = 5;
        num += 3; // Equivalent to num = num + 3;
        System.out.println("num: " + num); // Output will be "num: 8"
    }
}
```

> **Subtraction Assignment Operator (-=):** It subtracts the value on the right-hand side from the variable on the left-hand side and assigns the result to the left-hand side variable.

Main.java
```java
public class Main {
    public static void main(String[] args) {
        int num = 10;
        num -= 4; // Equivalent to num = num - 4;
        System.out.println("num: " + num); // Output will be "num: 6"
    }
}
```

> **Multiplication Assignment Operator (*=):** It multiplies the variable on the left-hand side by the value on the right-hand side and assigns the result to the left-hand side variable.

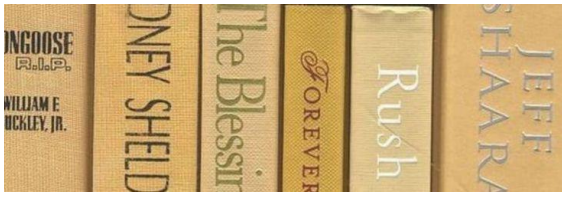
Main.java
```java
public class Main {
    public static void main(String[] args) {
        int num = 3;
        num *= 5; // Equivalent to num = num * 5;
        System.out.println("num: " + num); // Output will be "num: 15"
    }
}
```

> **Division Assignment Operator (/=):** It divides the variable on the left-hand side by the value on the right-hand side and assigns the result to the left-hand side variable.

Main.java
```java
public class Main {
    public static void main(String[] args) {
        int num = 20;
        num /= 4; // Equivalent to num = num / 4;
        System.out.println("num: " + num); // Output will be "num: 5"
    }
}
```

> **Modulus Assignment Operator (%=):** It computes the remainder when the variable on the left-hand side is divided by the value on the right-hand side and assigns the result to the left-hand side variable.

Main.java
```java
public class Main {
    public static void main(String[] args) {
        int num = 17;
        num %= 5; // Equivalent to num = num % 5;
        System.out.println("num: " + num); // Output will be "num: 2"
    }
}
```



Relational Operators

> Relational operators are used in programming to compare values and determine the relationship between them.
> In Java, there are several comparison operators that allow you to perform comparisons and make decisions based on the results.
> These operators are commonly used in conditional statements to control the flow of the program.

Comparison Operators

- **== (equal to):** Checks if two values are equal.
- **!= (not equal to):** Checks if two values are not equal.
- **> (greater than):** Checks if the left operand is greater than the right operand.
- **< (less than):** Checks if the left operand is less than the right operand.
- **>= (greater than or equal to):** Checks if the left operand is greater than or equal to the right operand.
- **<= (less than or equal to):** Checks if the left operand is less than or equal to the right operand.



Boolean Operators

> Boolean operators, also known as logical operators, are used in programming to combine and manipulate boolean values (true or false).
> In Java, there are three main boolean operators: AND (&&), OR (||), and NOT (!).
> These operators allow you to build complex conditions by combining multiple boolean expressions.

Logical Operators

> **AND (&&):** The AND operator returns true if both of its operands are true, and false otherwise.
> **OR (||):** The OR operator returns true if at least one of its operands is true, and false otherwise.
> **NOT (!):** The NOT operator reverses the logical state of its operand. If the operand is true, the NOT operator returns false, and if the operand is false, it returns true.